CPSC-559 Group 1

Final Report

Avery Keuben - 30170731

Braedon Haensel - 30144363

Brian Heckel - 30165243

Pranab Mainali - 30110321

Taylor Wong - 30139663

Contents

Introduction

This project implements a distributed online checkers platform designed to explore and apply core concepts in distributed systems to the classic game checkers, such as replication, leader election, synchronization, consistency models, and fault tolerance.

At a high level, the system is composed of four main process types, a browser based client, a Name Server, a cluster of Matchmaker servers, and a set of Game Servers.

The client is a React.js web application that allows users to join a matchmaking queue, be paired with an opponent, and play real-time checkers matches through a WebSocket connection to a Game Server.

The Name Server acts as a simple, dynamic service registry that keeps track of all currently active Matchmaker and Game Server instances so that components can discover each other at runtime.

The Matchmaker servers form a leader based cluster that manages the global matchmaking queue and the leaderboard, using a leader election protocol (based on the bully algorithm) and replication to provide a single logical service despite multiple underlying processes.

The Game Servers host individual games, maintain authoritative game state, validate moves, and coordinate game events such as draws and forfeits while replicating state so that another server can take over in case of failure.

The primary goals of the project are to:

- Design a modular architecture that clearly separates concerns between matchmaking and leaderboard ranking (Matchmakers), and gameplay (Game Servers).
- Apply a concrete leader election algorithm in a realistic setting and integrate it with request routing and replication.
- Experiment with different consistency and synchronization strategies tailored to each subsystem's requirements, rather than using a single one-size-fits-all approach.
- Implement practical fault detection and recovery mechanisms so that the system can tolerate process crashes without corrupting shared state.

The remainder of this report describes the architecture of the system, the chosen consistency and synchronization models, and the mechanisms used to achieve fault tolerance.

We also discuss key design trade offs, limitations, and potential improvements that could be made in future iterations of the system.

Architecture

Our system includes 4 main process types, the client, the Name Server, the Matchmaker, and the Game Server.

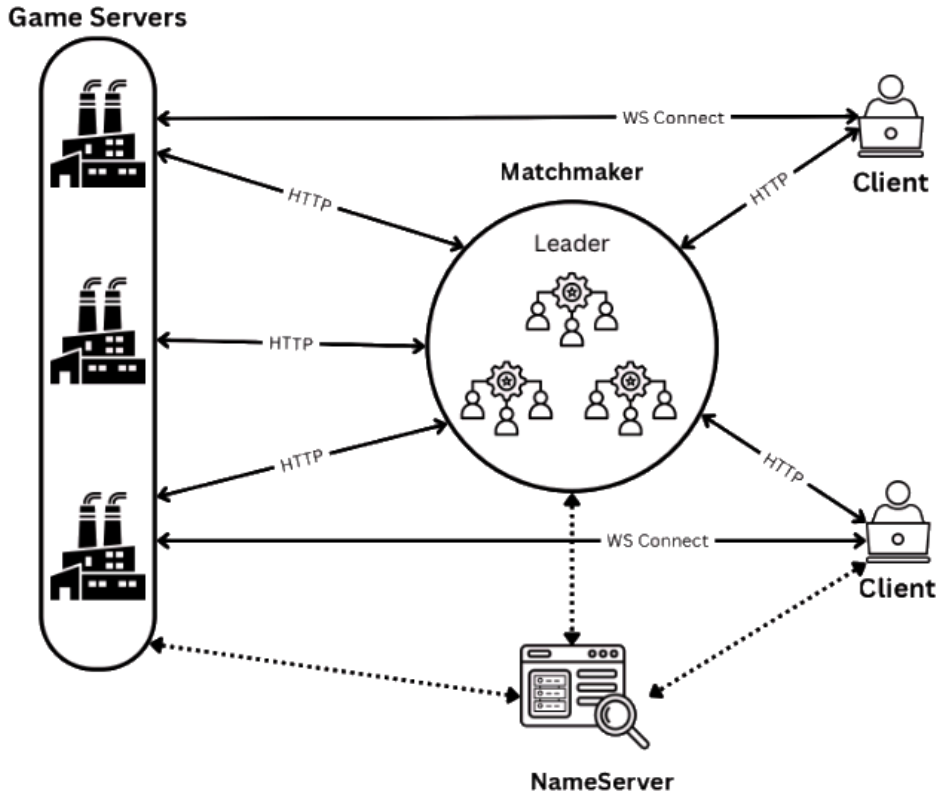


Figure 1: Overview of the System Architecture

The Client

The client is a React.js app the user can access in their web browser. It enables end users to initiate and play a game of checkers with an opponent. This software handles all user interaction as well as communication with the other server types.

The Name Server

The Name Server is intended to act primarily as a DNS server. In a real deployment, we would just use DNS, but while actively testing, it was more convenient to have a server that would dynamically register servers as they spawned or died. We ensured the server was as simple as possible in order to reduce the chance of failure to the point where it is primarily returns a list of addresses. As its only function is to return a list of servers connected to the system, it theoretically could have been just implemented as a hard-coded list included in every other process, but we chose to do it this way in order to ease development.

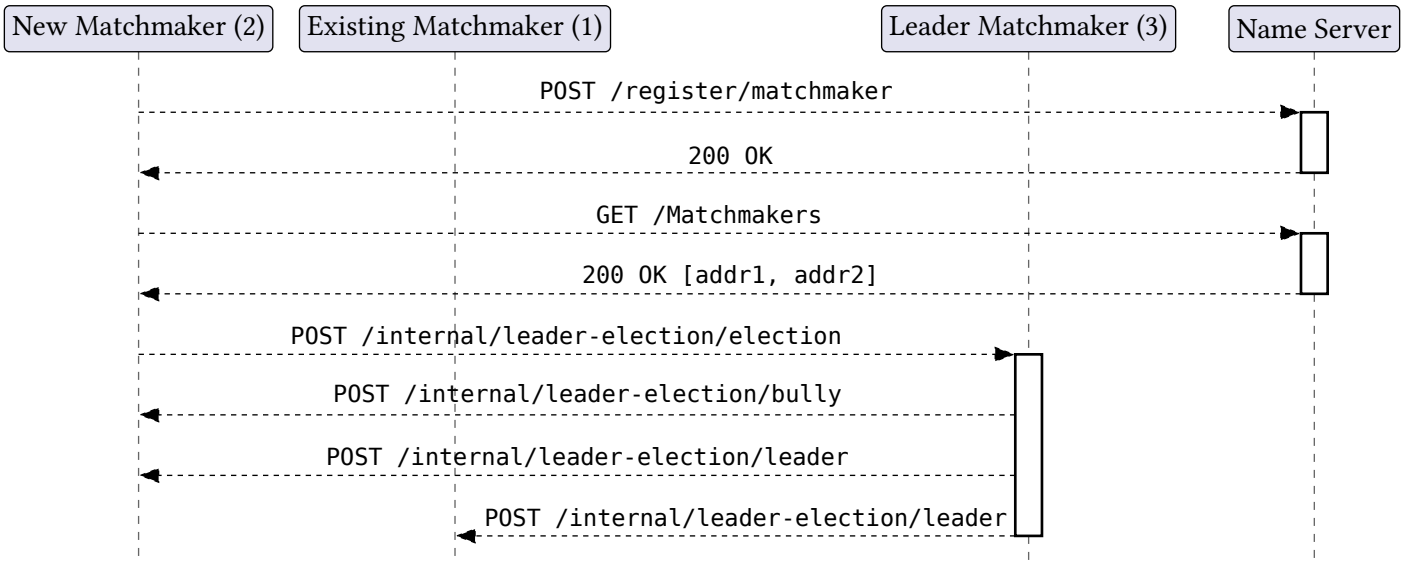
Name servers have the following API surface:

Endpoint	Description
`GET /matchmakers`	Returns a list of currently active Matchmakers
`GET /game-servers`	Returns a list of currently active Game Servers
`POST /register/matchmaker`	Register a new Matchmaker. Called by the new Matchmaker upon the process starting.
`POST /register/game-server`	Register a new Game Server. Called by the new Game Server upon the process starting.
`POST /deregister/matchmaker`	Deregister a current Matchmaker. Called by another process if the given Matchmaker is determined to have crashed.
`POST /deregister/game-server`	Deregister a current Game Server. Called by another process if the given Game Server is determined to have crashed.

Matchmakers

Matchmakers are a leader-based cluster of servers using the bully algorithm. Their primary role is to facilitate pairing clients for games as well as allocating Game Servers for such games. They also have the secondary function of storing the leaderboard for the system. Players are assigned a wins and loss count based on the result of each game.

Upon spawning, the new Matchmaker will first register itself with the Name Server, then initiate an election, at which point it will request the list of other Matchmaker servers.



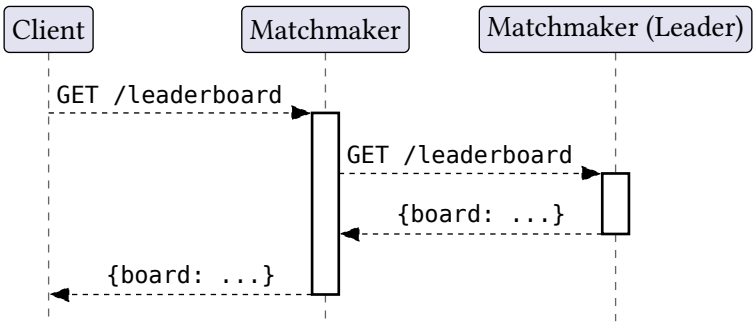
We included an additional synchronization mechanism into the leader election protocol, which will be further discussed in the Synchronization section.

Matchmakers have the following public API service:

Endpoint	Description
`GET /leaderboard`	Returns the current leaderboard state as a JSON object.
`POST /queue/add`	Add a client to the Matchmaker queue. Expects clients to frequently poll the Matchmaker for updates to their queue status.
`POST /queue/poll`	Check a clients position in the queue. Returns one of: `not in queue`, `in queue`, or `found match`
`POST /queue/leave`	Forfeit position in queue. If a client fails to send this, they will still be removed after not polling for a while.
`POST /match/request-new-game-server`	Ask the Matchmaker for a new server to host the game for a client. This will be called if a client is disconnected from a Game Server.
`POST /match/updateleaderboard`	Called from the Game Server to ask the Matchmaker to update the leaderboard with a new game result.
`POST /match/end`	Called by the Game Server when a match ends. Removes the match from the list of known matches.

There are additional endpoints for leader-election as well as data replication, but they are omitted in this section.

Each Matchmaker acts as a reverse proxy to the leader server. This way, clients or Game Servers do not need to be concerned with contacting the leader server, rather, they can send the message to any Matchmaker, and it will be forwarded to the leader server.



Game Server

The Game Server's primary purpose is to run games between players. All communication takes place primarily over web-sockets to facilitate push messaging instead of requiring clients to poll for updates to the game state. Game servers were separated from Matchmakers to establish a differentiation of concerns, and to ensure that a single queue exists on the leader server, which can then perform load balancing across all of the Game Servers.

A game is started by a Matchmaker informing the server of a new game that will take place on that server. Once both clients join, the game is started, and web-sockets are used to facilitate communication between the clients. The server is responsible for checking the validity of a move from a client, determining whether a game has ended, as well as handling and forwarding draw requests and forfeits.

Game servers back up their current game state by informing all other Game Servers of the new state on any write to the game state. This way, if a Game Server were to go down, then another Game Server will be able to take ownership of the game (also determined by the Matchmaker). This synchronization and consistency model will be further discussed in the corresponding sections, Synchronization and Consistency.

A Game Server has the following API surface:

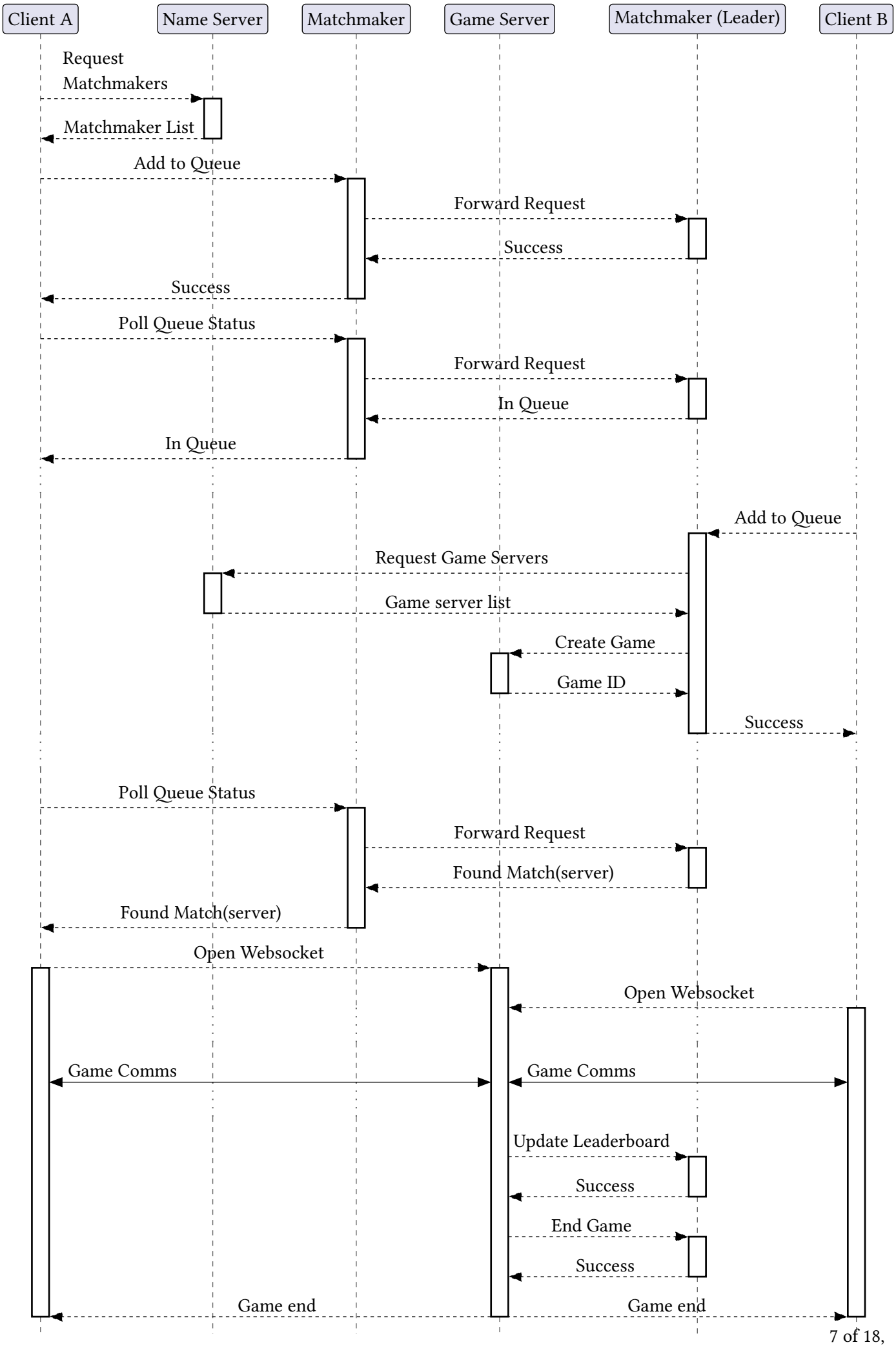
Endpoint	Description
`POST /newGame`	Create a new game on the server. Called by a Matchmaker
`POST /takeover`	Request that the target server becomes the new leader for the given game. Used by the Matchmaker when a Game Server goes down.

Example

The following is all actions that are taken during regular usage of the application.

1. Client requests the list of Matchmakers from the Name Server
2. Client contacts a Matchmaker (not necessarily the leader) to request that they be added to the queue
    1. If the contacted server is not the leader, then the message is forwarded
3. Client periodically polls the server to check their queue status
4. Another client joins the queue
5. Game server starts a new game
6. Client polls the server, and learns the Game Server address to connect to for the new game
7. Client opens a web-socket connection to the Game Server
8. Game Messages are exchanged
9. Game server sends update leaderboard message to Matchmaker
10. Game server sends end game message to Matchmaker
11. Clients disconnect from the Game Server

The following is a sequence diagram showing how this ties together with all the services:


Client A
Name Server
Matchmaker
Game Server
Matchmaker (Leader)
Client B
Request Matchmakers
Matchmaker List
Add to Queue
Forward Request
Success
Success
Poll Queue Status
Forward Request
In Queue
In Queue
Add to Queue
Request Game Servers
Game server list
Create Game
Game ID
Success
Poll Queue Status
Forward Request
Found Match(server)
Found Match(server)
Open Websocket
Open Websocket
Game Comms
Game Comms
Update Leaderboard
Success
End Game
Success
Game end
Game end

The following messages can be exchanged during the Game Communication section:

Message Type	Message Parameters	Description
`join`	`{ "user": string }`	Lets the Game Server know which client is connecting
`registered`	`{ }`	Lets the client know the server has acknowledged the `join` request
`start`	`{ "player_color": 'black' | 'red', "opponent": string }`	Lets the client know the game has started and assigns the player their color and informs them of their opponent's name
`move`	`{ "source_index": number, "destination_index": number }`	Message from the client to the server informing of a move that was made from the client.
`ùpdate_state`	`{ "tile_states": TileState[] "turn": PlayerColor "previous_moves": { "source_index": number "destination_index": number }[] }`	Informs a client about the current game state. Is sent after a new move is made, or if the previous move was deemed invalid.
`game_end`	`{ "winner": 'red' | 'black' | 'draw' }`	Informs the client that the game is over. The client is free to disconnect without penalty after they receive this message.
`forfeit`	`{ }`	Informs the server that the sender wishes to forfeit the game. Immediately ends the game, and a `game_end` message is sent.
`request_draw`	`{ }`	Is sent by a client to request that the game ends in a draw. One of these messages must be received by both players before the game ends. Any move will invalidate the draw request for both players. If this message is received by a client, it indicates the other party has initiated a draw request.

Synchronization

Synchronization in our system is responsible for keeping replicated state aligned across processes so that any node can safely take over work from a failed peer without users noticing inconsistent behavior.

We focus on two synchronization domains, the Matchmaker cluster (queue and leaderboard state) and the game Servers (per game board state).

Matchmaker Synchronization

As described in the architecture section, Matchmakers form a leader based cluster and store two main pieces of shared state, the matchmaking queue and the global leaderboard.

To keep this replicated state synchronized, we route all logically "write" operations through the current leader and then broadcast the resulting updates to all follower Matchmakers in a consistent order.

Concretely, all external clients and Game Servers are allowed to send their HTTP requests to any Matchmaker instance.

If the contacted node is not the leader, it acts as a reverse proxy and forwards the request to the current leader over an internal API, then relays the leader's response back to the caller.

For example:

- `POST /queue/add`, `POST /queue/poll`, and `POST /queue/leave` all flow through the leader, which maintains the single authoritative matchmaking queue and then pushes the updated queue state to all other Matchmakers.
- `POST /match/updateleaderboard` and `GET /leaderboard` are handled by the leader, which applies leaderboard updates in a total order and replicates that updated leaderboard state to its peers.

This "forward-to-leader then broadcast" pattern provides sequential consistency for the queue and leaderboard. Every Matchmaker observes the same sequence of updates, in the same order, even though clients may contact different Matchmaker endpoints

Overall, the Matchmaker synchronization mechanism can be summarized as:

- All writes go through a single leader.
- The leader broadcasts ordered updates to followers.
- Followers use these broadcasts both to keep state up-to-date and to detect crashed peers.
- Leader election includes a synchronization phase to ensure a newly elected leader starts from a consistent state.

Game Server Synchronization

Game servers are responsible for the state of individual checkers games, and we replicate this state so that other servers can take over seamlessly if the primary game Server for a game fails.

Each active game is assigned a single Game Server and all moves, forfeits, and draw requests for that game flow through that leader.

Whenever the Game Server applies a write to the game state (for example, accepting a move from a client or ending the game), it immediately broadcasts the updated state to all other Game Servers.

In practice, this means that after each change in the game, the server pushes the current board configuration, whose turn it is, and any relevant history or metadata to its peers.

This synchronization scheme has several important properties:

- **Per game total order:** Because all reads and writes for a game go through that game's Game Server, the server sees a single, linear order of events for that game, which it can then apply and replicate in the same order to other servers.
- **Relaxed ordering across games:** The ordering of events between different games does not matter, so long as each game's internal sequence of moves remains consistent.

- **Takeover readiness:** At any point in time, another Game Server has a recent copy of the game state, and the Matchmaker can instruct that server to take over hosting the game if the original Game Server crashes.

Client Visible Effects of Synchronization

From the clients' perspective, this synchronization strategy is intentionally hidden:

- Clients connect to any Matchmaker endpoint and always observe a single, coherent queue and leaderboard, even though multiple Matchmakers exist behind the scenes.
- During gameplay, clients connect to a single Game Server over WebSockets and see a consistent game state. If the underlying Game Server fails, the Matchmaker can transparently instruct a different server to take over using the replicated game state.

By combining leader based replication at both the Matchmaker and Game Server layers, our system's synchronization model supports the consistency guarantees described in the Consistency section while directly enabling the fault detection and recovery mechanisms detailed in the Fault Tolerance section.

Leader Election

Our Matchmaker service uses the *bully algorithm* to come to a consensus on which Matchmaker will serve to be the leader and handle all forwarded requests from clients. We chose the bully algorithm due to its simplicity in design, ability to handle new servers joining the distributed system dynamically, and natural tolerance of server faults. We utilize the Name Server to produce ID's for each Matchmaker that joins the queue, so that each Matchmaker will have a unique ID such that the Matchmaker with the highest ID is the leader. Our reasoning for not using a quorum based leader election model is due to the assumption that all network channels are reliable. Since it is assumed that each network channel cannot fail, we consequently assume that all Matchmakers are able to communicate with each other. Thus, we cannot have a scenario where two Matchmakers both believe that they are the leader due to an inability to communicate with each other during the leader election.

Initiating Leader Elections

On startup, servers will start a leader election and update their list of known servers using a request to the Name Server. Then, if the server has the highest ID, it will declare itself as the leader and send `leader` messages to the other Matchmakers to confirm that they are the leader. Otherwise, if there are other Matchmakers with a higher ID, then the Matchmaker will send an `election` message with their ID to indicate that an election is starting. The Matchmaker will then wait for a bully message via a timeout, and if the Matchmaker is not bullied, then it will declare itself as the leader and send `leader` messages to all other servers to indicate that it is the leader. Alternatively, if the Matchmaker receives a `bully` message during the timeout, then it will wait for a leader message to be received during a new timeout window. If a leader message is not received within the timeout, then it will initiate a new leader election.

The following is the source code after a server has initiated an election and waits for a bully or for the timeout period to end.

```
select {
case <-m.bullyTimer.C:
  // Timer fired, so no bully() responses received in time. Declare itself leader
  m.leaderMu.Lock()
  log.Println("No bully() responses received. Declaring itself leader with ID:", m.ID)
  m.Leader = Server{
    ID:  m.ID,
    URL: m.URL,
  }
  m.leaderMu.Unlock()

  // Get the latest data from all Matchmakers, then send leader messages with it
  m.SynchronizeDataAndSendLeaderMessages()

case <-m.bullyTimerChan:
  // Bullied before the timer fired. Wait for a leader message
  log.Printf("Received a bully() response. Waiting up to %dms for a leader message\n",
    LEADER_ELECTION_TIMEOUT_SEC.Milliseconds())
  m.leaderTimer = time.NewTimer(LEADER_ELECTION_TIMEOUT_SEC)
  m.leaderTimerChan = make(chan struct{})
  select {
  case <-m.leaderTimer.C:
    // Timer fired, so no leader received. Something went wrong, so initiate
    // a new election
    log.Println("No leader responses received")
    m.InitiateElection()
  case <-m.leaderTimerChan:
    // Received a leader message. The message is handled by the leader
    // message handler, so return
    return
  }

}
```

Matchmaker servers also need to handle `bully`, `leader`, and `election` requests from other Matchmakers. When a Matchmaker receives a `bully` request, then it is because we currently tried to elect to be the leader, and a Matchmaker with a higher ID has bullied us out of being the leader. The Matchmaker will stop the bully timeout and close the `bullyTimeoutChan` to indicate that the Matchmaker has received a bully message in time, and will then wait for a leader message.

```
// Handle an incoming bully() Bully leader election message.
func (m *Matchmaker) HandleBullyRequest(w http.ResponseWriter, r *http.Request) {
  // Interrupt the bully timer so it never fires
  if m.bullyTimer != nil && m.bullyTimer.Stop() {
    if m.bullyTimerChan != nil {
      // Close the bully timer chan to notify the thread to stop waiting for the timer
      close(m.bullyTimerChan)
    }
  }
}
```

When a Matchmaker handles an `election` request from a Matchmaker with a lower ID than itself, it will send a `bully` request to the server, and initiate its own election unless it is already running in one. If the request came from a previously unknown Matchmaker, then it will update its list of known other Matchmakers to include the new Matchmaker.

When a Matchmaker handles a `leader` request, then we check if the Matchmaker was waiting for a `leader` message, and if so, we stop the leader message timeout. We potentially update our list of other Matchmakers if the message came from an unknown Matchmaker, and we set the new Matchmaker to be the `leader` and synchronize with the leader's data which is included in the request.

Consistency

As mentioned in the architecture section, our project has two replicated subsystems: the Matchmaking Servers and the Game Servers. Each uses a consistency model based on the requirements of their specific tasks and function.

Matchmaking Server Consistency Model (Sequential Consistency)

The Matchmaking Servers maintain shared state in the form of the player queue and the global leaderboard. For this component, we chose to adopt a sequential consistency model which is implemented by having all requests sent to the leader which in turn broadcasts the changes in a consistent order to all of the replicas.

The sequential consistency model ensures that all Matchmaking Servers observe identical queue states at all times, preventing anomalies such as players being matched multiple times or skipped due to divergent views of the queue. For example, under a weaker consistency model if two players (Player A and Player B) were to join the queue at the same time and there was already a player (Player C) in queue then one server could believe that the next game should be between Player C and Player A while another could believe that it should be between Player C and Player B.

While the Sequential Consistency model is more strict than necessary for the leaderboard as our current scoring system is commutative, it does provide the ability to switch to a non-commutative ranking system such as the ELO system where the order of game results does affect the final rankings.

Overall, we believe that the added cost of having the leader synchronize all updates is a reasonable sacrifice for the benefits a sequential consistency model provides.

Game Server Consistency Model (Sequential Consistency)

The Game Servers store the state of all active games and must keep these states consistent amongst the servers so that if a Game Server fails the others can take over.

The events of each game of checkers must be agreed upon by all Game Servers since in many cases moves happening out of order will lead to different or even illegal board states. However, the ordering of events between games is not important, so long as all updates are applied. For this reason we chose a more relaxed approach than that of the Matchmaker servers while still providing Sequential Consistency.

In our system, each game has one Game Server designated as its “leader” and only that server can update the game state for that game. The clients of the players in the game communicate directly with that server, which then broadcasts the updates to the other servers. Since the reads and writes for a game only happen at that game’s leader server, we can construct a total order of all the reads and writes that agrees with all local orders by simply chaining the local orderings together (any order of the local orderings will still form a valid total ordering).

This approach allows us to maintain consistent game states without requiring a central leader to sequence all of the reads and writes across separate games.

Fault Tolerance

Fault tolerance is a system's ability to continue operating correctly even when some of its components fail. In our system, the two main backend components subject to failure are the Game Server and the Matchmaker server, each with their own detection and recovery mechanisms.

Game Server Fault Tolerance

As mentioned in the architecture section, the Game Server's primary purpose is to run games between players. So it is more of a resource for higher that gets selected for use by the Matchmaker when it sees fit. Below is the detailed review of how the faulty Game Server is detected and is recovered from. In short a faulty Game Server is detected when it gets selected to be used or is currently being used, once detected that it cannot be used, the system simply moves to using one of the remaining Game Server resources instead while making sure the faulty server is no longer reachable in the system.

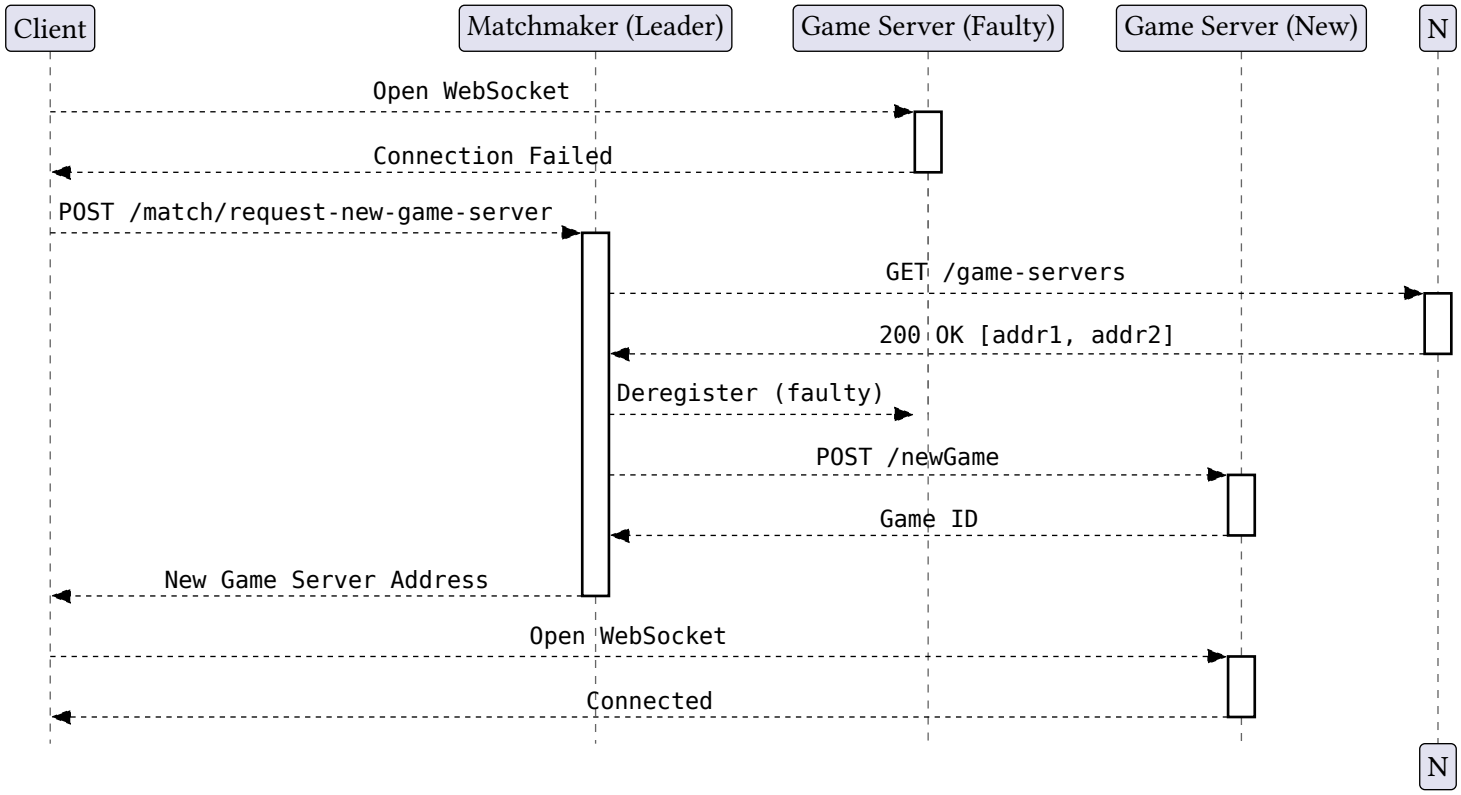
Detection

With that being said a faulty Game Server can be detected in four ways:

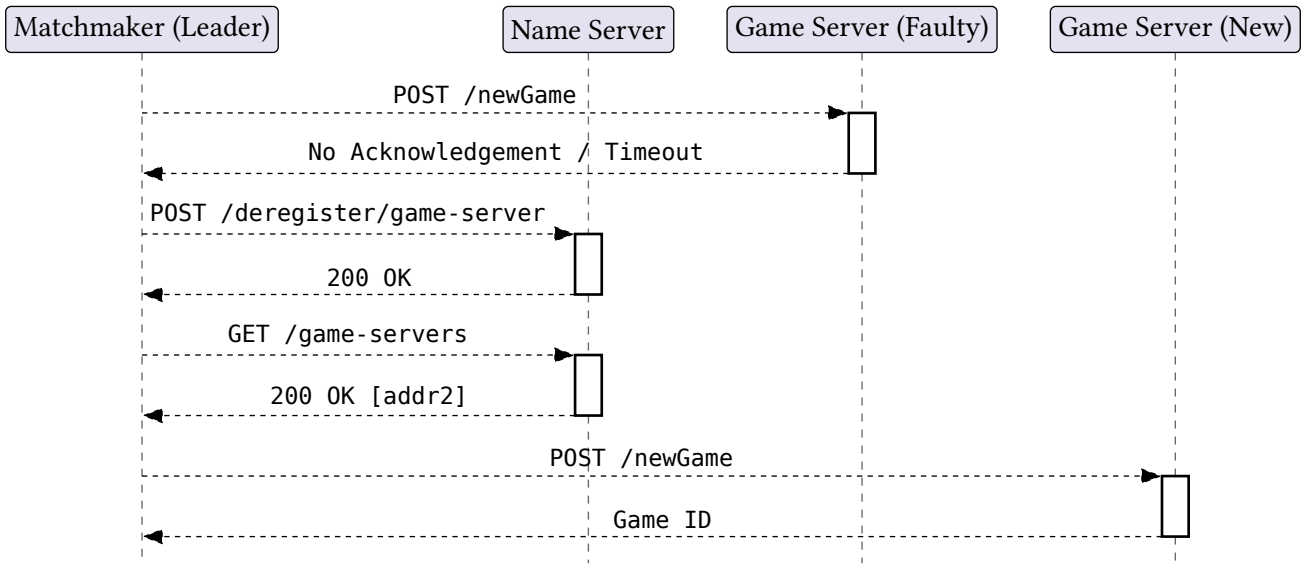
1. **Matchmaker acknowledgement timeout:** When the Matchmaker assigns a Game Server to host a game and does not receive an acknowledgement, it determines that the Game Server is faulty.
2. **Failed WebSocket connection:** When a client attempts to open a WebSocket connection to a Game Server and is unable to do so, the client infers the server is down.
3. **Broken WebSocket connection:** If a client already has an active WebSocket connection to a Game Server and it is severed due to a server failure, the fault is detected mid-game.
4. **Synchronization failure:** When Game Servers synchronize state with each other and one fails to send an acknowledgement, the remaining Game Servers detect the fault.

Recovery

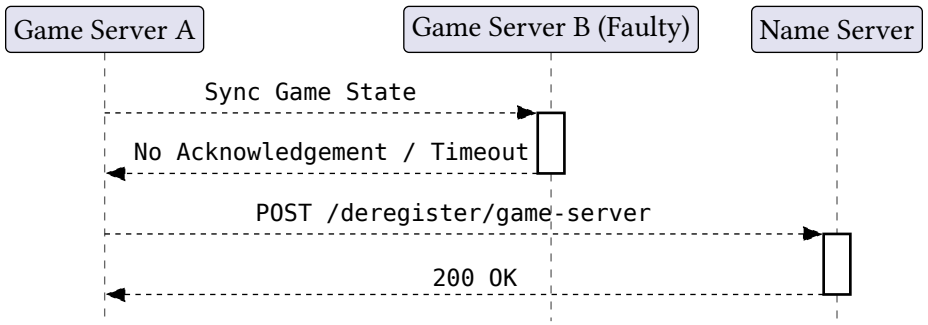
- If a client cannot open or loses a WebSocket connection, it contacts the Matchmaker to request a new Game Server to continue the game.



- When the Matchmaker detects a faulty Game Server, either directly or upon receiving such a report from a client, it deregisters the server and assigns the game to a new available Game Server.



- When a Game Server detects a faulty peer during synchronization, it similarly deregisters the faulty server on the nameserver.



Matchmaker Fault Tolerance

As mentioned in the architecture section, the Matchmakers are a leader-based cluster of servers using the bully algorithm. Their primary role is to facilitate pairing clients for games as well as allocating Game Servers for such games. Both Game Servers and clients will be making requests to Matchmaker and when they do, they may use any endpoint to a Matchmaker in the cluster and will not know the leader specifically. Therefor it is up to the internal communication protocols within the cluster to determine fault and recover based on which specific server is faulty.

In short how fault tolerance in Matchmaker works is that, from a Game Server and client perspective, they just keep trying the endpoints in the Matchmaker cluster until one succeeds. The cluster itself, through communication when synchronizing or running normal operation will knock out any servers that are faulty, and when the leader is faulty, they simply pick a new leader and resume.

Detection

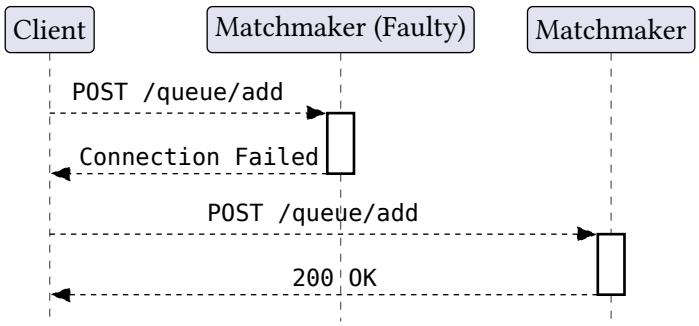
A faulty Matchmaker can be detected in four ways:

1. **Client or Game Server request failure:** When a client or Game Server sends a request to a Matchmaker and receives a `404` or connection error response, it concludes the Matchmaker has gone down.
2. **Synchronization failure:** Non-leader Matchmakers receive communications from the leader through broadcasts and communicate with each other during synchronization. If one of these internal messages fails, the sending Matchmaker infers that the target non-leader Matchmaker has crashed.
3. **Failed redirect to leader:** Every non-leader Matchmaker acts as a reverse proxy to the leader. If a non-leader receives a request and the redirect to the leader fails, it detects that the leader Matchmaker is down.
4. **Matchmaker Heartbeats:** The leader Matchmaker sends a periodic heartbeat (every 3 seconds) to all other non-leader Matchmakers. If the request to another Matchmakers receives a 404 or connection error response, it concludes the Matchmaker has gone down. Additionally, each non-leader Matchmaker periodically checks

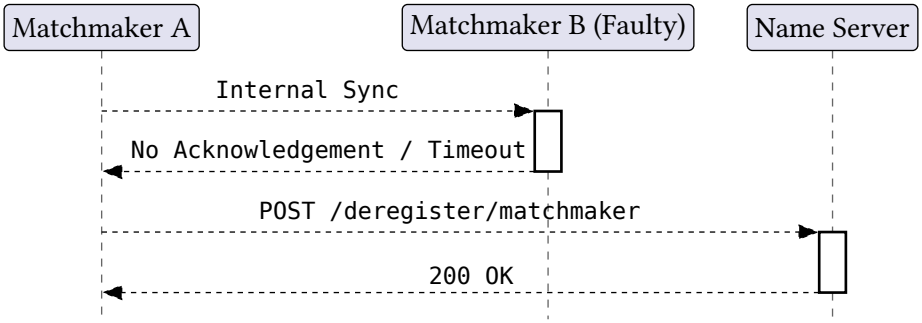
(every 3 seconds) if it has received a new heartbeat from the leader. If 3 consecutive checks fail, it concludes the leader is down, and will consequently initiate a new leader election to replace the down leader.

Recovery

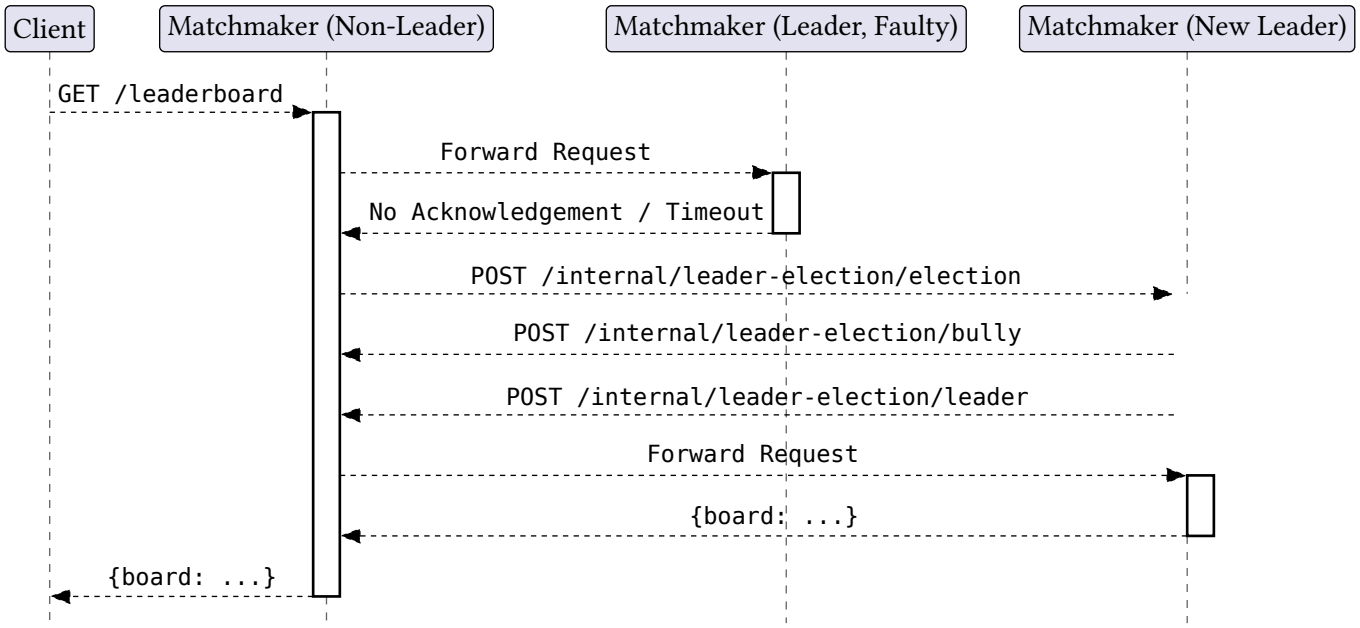
- In the case of a failed request, the client or Game Server simply retries the request against a different Matchmaker, ensuring no hard dependency on a single Matchmaker instance. Again the client or Game Server will not know of the leader.



- If a non-leader Matchmaker is detected as crashed — whether via synchronization or otherwise — it is deregistered from both the cluster and the Name Server, so no further requests are routed to it.



- If the leader Matchmaker is detected as down during a redirect or 3 consecutive failed heartbeats, a new leader election is initiated using the bully algorithm. The remaining non-leader Matchmakers elect a new leader and normal operation resumes with the initial request being forwarded to the new leader.



Additional recovery management implemented in the Matchmaker is the Matchmaker's ability to detect an unresponsive client and remove them from queue. Under current architecture the client makes a request to the Matchmaker to be added to queue. At this stage the client could have closed their browser or there was some fault.

The Matchmaker starts a 5 second timer when a client is added to queue which resets every time the client requests a queue poll. If there is no poll requests within the 5 seconds, the client gets removed from the queue.

Limitations

Name Server as a point of failure
While we designed the Name Server to be as simple and robust as possible, we do recognize that it serves as a single point of failure. If the Name Server were to fail at any point recovery is difficult as all other servers would need to restart after the Name Server is restarted.

Total Failure leads to game-state loss and loss of the queue
While both the Matchmaker and Game Server clusters are capable of recovering as long as a single replica is maintained, if all of the Game Servers or Matchmaker servers fail our system will lose some data. In the case that all of the Game Servers fail, all game state will be lost. If all of the Matchmaking servers fail, the queue will be lost, but a backup of the leaderboards is automatically saved to and can be restored from disk.

Automatic Trust of Servers
Our current design does not implement any level of security for server registration. This means that a malicious Matchmaking Server or Game Server could theoretically enter the system.

Relying on network
Our system currently relies on the network being reliable. If the network itself were to fail even temporarily, it is possible that a Matchmaker or Game Server would incorrectly be classified as failed and subsequently removed from the system.

Game Server deregistering another lagging Game Server when synchronizing
If a Game Server were to be under exceedingly high load, it is possible that it will fail to respond to another Game Server within the timeout window, which will cause the other Game Server to deregister it.

The system puts higher load on leader Matchmaker to ensure consistency
The structure of the Matchmaking cluster requires that **all** requests get processed by the leader, which puts a much higher load on it than the other Matchmakers. This in theory could lead to the leader slowing down and bottlenecking the system.